

Übung 10

Arbeiten mit git

Digitale Technologien - Cloud Computing

4abHWII

March 20, 2026



Übungsleiter: Albert Greinöcker

Ziel der Übung:

- Anlegen eines github accounts
- Kennenlernen der grundsätzlichen Befehle und Funktionen von git
- Zusammenarbeiten in einem git-Projekt
- Umgang mit Forks und Pull Requests

1 Anlegen eines github-Accounts und eines leeren Projektes

1.1 Account anlegen

Unter <http://github.com> soll ein Account angelegt werden (falls das nicht schon passiert ist).

1.2 Anlegen eines git-Projektes

a. Erstellen eines leeren Repositories:

```
1 mkdir python-uebung
2 cd python-uebung
3 git init
```

b. Erstelle eine beliebige Python-Datei, z.B. `calculator.py`:

```
1 def add(a, b):
2     return a + b
3
4 if __name__ == "__main__":
5     print("5 + 3 =", add(5, 3))
```

c. Änderungen übernehmen:

```
1 git add calculator.py
2 git commit -m "Add simple calculator program"
```

1.3 Repository über die Website erzeugen

Bis jetzt haben wir ja nur ein lokales repository, wir brauchen eines auf github:

- Es soll ein Python-Projekt werden, deshalb bei `.gitignore` Python auswählen.
- `public`, damit die untenstehenden Übungen möglich werden. Wenn man `private` einstellt, dann ist es nur für den aktuellen user sichtbar.
- Ein `readme` soll erstellt und grundlegend editiert werden (zur Erinnerung: Die Sprache dafür ist markdown)
- Lizenz: Für ein Projekt zum Ausprobieren ist es eigentlich egal, aber die Übersicht auf den Folien dazulesen.

Nach dem Anlegen ist die clone URI dann unter Code sichtbar.

1.4 Lokales Projekt verknüpfen

```
1 git remote add origin <dein-github-repo-link>
2 git branch -M main
3 git push -u origin main # Falls nicht klappt statt main ein anderer Name
```

1.5 Neue Inhalte einfügen und auf github pushen

a. in das lokale repository einfügen:

```
1 git commit -m "Add subtraction function"
```

b. auf github geben:

```
1 git push -u origin main
```

1.6 Datei ändern

Die hochgeladene Datei bitte nochmals ändern, also z.B. eine neue Funktion hinzugeben. Diese nochmals committen und pushen.

Einsehen der Änderung: Auf der Homepage des repositories sollte die Datei sichtbar sein. Rechts oben ist ein Button `history` der alle Änderungen genau anzeigt.

mit dem Befehl `git log <dateiname>` können diese über die Konsole eingesehen werden.

1.7 Zusammenarbeiten im Team

Um es zu ermöglichen dass man in einem Projekt zusammenarbeitet muss man Collaborators zum Repository hinzufügen. Das geht unter Settings => Collaborators => Manage Access => Add People. Dort den Benutzernamen eingeben.

Ist man Collaborator eines Projektes kann man mit `git clone` das Projekt lokal holen, Änderungen machen oder neue Dateien hinzufügen und diese auf github hochladen.

Das bitte mit dem repository des Nachbarn ausprobieren. Die Befehle wie man z.B. die Änderungen des collaborators lokal übernehmen kann sind den Folien zu entnehmen.

1.8 Mit fremden Projekten arbeiten (Fork und Pull Request)

Möchte man ein Fremdes Projekt verwenden kann man dies Forken und die Änderungen mittels Pull requests vorschlagen. Versuche das mit einem Repository in der Klasse bei dem man nicht Collaborator ist. Dort sind Änderungen einzubringen wenn man im eigenen Repository Änderungen macht.

a. Projekt forken: Klicke auf der Website im Ziel-Repository auf Fork.

b. Deinen Fork klonen:

```
1 git clone <dein-fork-link>
2 cd <repo>
```

c. Branch erstellen, Änderungen vornehmen und pushen:

```
1 git checkout -b fix-bug-xyz
2 # Änderungen am Code vornehmen
3 git add .
4 git commit -m "Fix Bug XYZ"
5 git push -u origin fix-bug-xyz
```

d. Auf GitHub:

- In deinem Fork auf Pull Request klicken.
- Ziel-Repository und Branch auswählen (oft automatisch erkannt).
- Beschreibung möglichst genau ergänzen und Pull Request absenden.

e. Im Ziel-Repository sollte nun unter Pull Requests ein neuer Eintrag sein. Diesen kann man übernehmen oder auch nicht.

2 Hinweise zur Passwortverwaltung

Man verwendet nicht das eigentliche github-Passwort für die oben beschriebenen Aktionen sondern einen Access token. Dieser wird folgendermaßen erstellt:

a. Auf das user-Icon rechts oben, dort Settings wählen

b. Unter Developer Settings ganz links unten Personal Access Tokens auswählen, dann rechts oben: Generate new Token (classic)

c. Bezgl. scopes: Hier sollte repo reichen.

Mit diesen Zeilen werden die credentials für eine gewisse Zeit gespeichert:

```
1 git config --global credential.helper cache
2 git config --global credential.helper 'cache --timeout=3600'
```

3 Hinweise zur Abgabe

Bitte die einzelnen Ergebnisse mittels einfachen Screenshots grundlegend dokumentieren